TP: Jeux de réflexion

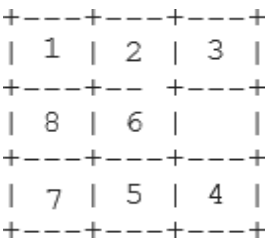
Le but de ce TP est de programmer le jeu du taquin en utilisant les algorithmes de recherche classique.

Jeu de taquin

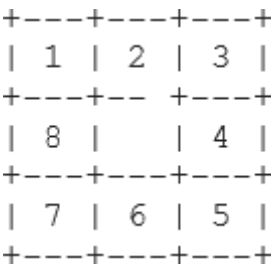
Dans ce TP, nous utiliserons un taquin "3 × 3", dont les cases sont numérotées de 1 à 8. Le jeu consiste à remettre dans l'ordre ces cases à partir d'une configuration initiale quelconque.

Exemple :

Etat initial



Etat final



Pour résoudre ce problème, nous utiliserons pour ce TP les algorithmes de recherche en profondeur et en profondeur limité. En effet, l'espace des situations du taquin forme un espace discrétisé représentable par un graphe. Et, d'autre part, le taquin se joue seul.

Objectif :

Ecrire un programme en Python permettant de jouer au taquin 3*3 en utilisant l'algorithme de **recherche en profondeur**, en **profondeur limité, en largeur** et en A*.

Principe :

En IA, pour résoudre un problème simple et simplement, à partir d'une situation donnée, il suffit de générer les situations possibles directement accessibles, de

vérifier si le but fait partie de ces nouvelles situations. Si le but recherché n'en fait pas partie, alors on prend une des situations générées et on répète le processus.

Pour un problème donné, il faut donc pouvoir le modéliser en situations et actions.

Les situations (ou états) sont aussi appelés **nœuds** car à partir de la **situation initiale (S0),** un arbre des situations futures possibles est généré.

Algorithme de recherche en profondeur (DFS) et en largeur (BFS):

L'algorithme est simple, on utilise 2 listes principales :

- La liste des noeuds à examiner, et dont il faut extraire les descendants directs si possible, c'est la liste **freeNodes** (les noeuds libres),
- La liste des noeuds déjà examinés et dont on a déjà extrait les descendants, il s'agit de la liste **closedNodes** (les noeuds clos). On doit chaque fois vérifier qu'un nœud à examiner n'existe pas dans la liste **closedNodes**

A partir de la première situation libre (**firstState**), on génère une liste des nœuds que l'on peut appeler **generatedStates**.

Parmi ces nœuds générés, on ne garde que ceux qui ne sont pas déjà dans les listes des nœuds à traiter (**freeNodes**) ou des noeuds déjà traités (**closedNodes**).

Les nouveaux restants (dans **generatedStates**) seront ajoutés à la liste des noeuds à traiter (**freeNodes**) :

- Ils sont ajoutés en tête de cette liste puisque la recherche s'effectue en profondeur (**DFS : Depth First Search**). Le parcours en profondeur utilise une pile (LIFO), où le dernier sommet arrivé dans la pile est le premier à en sortir.

-Ou ils sont ajoutés en queue de cette liste si la recherche s'effectue en largeur (**BFS : Breadth First Search).** Le parcours en largeur utilise une file d'attente (FIFO), où le premier sommet arrivé dans la file est aussi le premier à en sortir.

```
procedure recherche DFS
freeNodes←{s0}
closedNodes←∅
goalNode←∅
success←false
While freeNodes≠∅A¬succes do
```

```
  //retirer la première situation libre et la placer dans les nœuds clos (déjà exploré)
  firstNode←firstOf(freeNodes)
  freeNodes←freeNodes\{firstNode}
  closedNodes←closedNodesU{firstNode}
  //générer les états directement accessibles à partir du noeud retiré
  generatedStates←successorsOf(firstNode)
  //ne garder que ceux non présents dans les noeuds libres et noeuds clos
  generatedStates←{s|s ∈ generatedStatesA(s ∉ freeNodes ∨ s ∉ closedNodes)}
  For all s ∈ generatedStates do
      evaluate(s)
      If s.value=goal Then
        success←true
        goalNode←s
      End If
  End For
  //en profondeur d'abord, ajout des noeuds en tête des noeuds libres
  //en largeur d'abord, ajout des noeuds en fin de liste
  If depthFirst=true //en profondeur
    Then freeNodes←generatedStatesUfreeNodes
    Else //en largeur
    freeNodes←freeNodesUgeneratedStates
End While
Fin procedure
```

Dans le cas de recherche en profondeur limité on doit ajouter l'argument L qui indique la limite de profondeur d'exploration (**recherche (s0, goal, depthfirst, L**)).

Indications:

Vous aurez besoin de :

- **t** : tableau de dimension [3][3] qui représente le taquin.
- **etat_depart :** indique l'état initial du jeu.
- **estEtatFinal** : retourne **True** si l'état passé en paramètre est l'état final.
- **transitions** : retourne la liste des états voisins pour un état donné. On suppose que le coût de chaque transition est de 1.

- **-position_case_vide (t)** qui renvoie la position de la case vide dans le taquin t sous la forme d'un couple (x,y) : x est le numéro de colonne, y est le numéro de ligne (les numéros débutant à 0).
- **-numero(t, x, y):** retourne la valeur qui existe dans la case de coordonnées (x,y).
- **permuter (t, c1, c2):** permet de permuter les pièces situées dans les cases de coordonnées c1 et c2. (c1=(x1,y1))
- **afficher_taquin (t) :** affiche le taquin dans l'état actuelle.

Travail demandé:

1) Ecrire un programme qui permet de résoudre le jeu de taquin en appliquant la recherche en profondeur d'abord.
   Afficher le nombre des états générés (tous les états possibles), 1e nombre des nœuds visités et le temps d'exécution de l'algorithme.
2) Appliquant la recherche en profondeur limitée tel que L=3.
   Si le résultat n'est pas trouvé au profondeur 3, incrémenter L jusqu'à trouver la bonne valeur.
3) Changer le programme en appliquant la recherche en largeur d'abord.
4) Implémenter une solution A* pour résoudre le taquin.
   On utilise comme fonction heuristique :
   f=g+h, avec :
   - g : nombre de jetons déplacés
   - h: nombre de jetons mal placés

5) Analyser les résultats obtenus pour chaque algorithme de recherche.
6) Refaire la question 5) en changeant l'état initial (prenant par exemple un taquin avec 5 cases mal placés).

Remarque:

- On crée tout d'abord une version en mode texte. Dans ce cas, on affiche sur la console le taquincourant jusqu'à arriver au taquin final.
- On veut afficher pour chaque méthode de recherche appliqué le nombre total des états générés etle nombre des nœuds clos.
- On doit enregistrer à chaque fois le liste des configurations de taquin (nœud de graphe) déjà visitéspour ne les traiter pas de nouveau.

7) Donnez une résolution de jeu de taquin 3*3 en mode graphique.